

Préambule

Le présent document suppose que vous avez déjà consulté le *cahier des charges* du client, et le document de *cadrage général* (« Organisation générale »). Comme cela y est expliqué, votre mentor (chef de projet senior) a défini plusieurs *sous-lots* et préparé divers artefacts pour vous mettre le pied à l'étrier. Vous allez appliquer ses consignes générales, et notamment suivre (dans l'ordre !) les différentes *phases* prévues pour cette *itération* et qui sont rappelées dans ce document. Mais auparavant, vous allez découvrir tous les éléments qui vous sont fournis.

Découverte (rapide) de l'infrastructure logicielle fournie

Téléchargez l'infrastructure logicielle qui est mise à votre disposition et consultez rapidement son contenu. Elle comporte un package principal (opinion) contenant notamment l'API¹ imposée par le client (interface ISocialNetwork) et une version vide de la classe à implémenter (SocialNetwork), les exceptions prévues par l'API (cf package exceptions) et une infrastructure de test (package tests). Le package hmi ainsi que la classe opinion.HMI permettent d'utiliser SocialNetwork via une interface graphique rudimentaire.

Installation de l'environnement de développement

Commencez d'abord par vérifier que vous avez bien tout récupéré :

- Créez un projet Java dans Eclipse.
- Importez-y tous les fichiers de l'infrastructure logicielle.
- Lancez le test intégré (tests.SocialNetworkTest) : il doit bien évidemment vous signaler des erreurs (car SocialNetwork est une coquille vide pour l'instant), mais il ne doit y avoir ni erreur de compilation (cf messages Eclipse), ni de problème à l'exécution. Si ce n'est pas le cas, il faut y remédier.
- Vous pouvez aussi rapidement essayer l'interface graphique (opinion.HMI) fournie par BibTel : les menus sont prêts, mais sont bien sûr inopérants à ce stade puisque les méthodes qu'ils appellent ne font rien pour l'instant. Cette interface graphique compatible avec l'API ne sert que pour illustrer² le fonctionnement en conditions réelles du produit après que celui-ci ait été validé. Elle vous permettra de faire des essais au cas par cas quand le besoin s'en fera sentir, mais votre démarche globale s'appuiera essentiellement sur des tests automatisés (comme ceux que vous avez lancés à l'étape précédente).

Il convient maintenant de créer une espace projet sur *GitLab*. Un seul des équipiers fera cette démarche, et y invitera ses coéquipiers (rôle « développeurs »), sans oublier le rapporteur du projet. Le dépôt sera initialisé avec cette première version du projet, et tous les autres devront cloner cette première version dans leur propre environnement de développement. Ça y est, vous êtes prêts à travailler tous ensemble !

1 Application Programming Interface

2 NB : l'interface définitive d'utilisation sera d'ailleurs bien plus aboutie... mais ce n'est pas votre problème.

Phase 1 – Spécifier les objectifs de l'itération

La spécification des objectifs de l'itération se fait typiquement en déterminant *l'API* de ce qui doit être développé... ce qui a déjà été décidé par votre mentor : pour cette première itération (*lot1.1*), votre cible sera l'implémentation des fonctionnalités permettant de gérer les *membres* du réseau social. Cela concerne spécifiquement les méthodes `addMember()`, `nbMembers()` et `toString()` de l'API imposée par le client.

Phase 2 – Préparer l'étape de validation de l'itération

Lorsqu'on aborde le développement d'un produit, qu'il soit logiciel ou non, il est impératif d'en déterminer à l'avance les modalités de recette. Il s'agit d'établir comment on pourra, le moment venu, mesurer le degré de conformité du produit livré à la demande client³. C'est pourquoi le principe de l'approche « *développement dirigé par les tests* » prévoit de définir des *scénarios de tests* et à développer le *code de test* associé *avant* de passer au développement proprement dit.

L'objectif de cette deuxième phase est donc de préparer la validation du (de la version de) produit que vous allez créer dans la suite de l'itération. **Cette validation se fera exclusivement à partir de l'API du produit, et non avec des essais interactifs via l'interface graphique.** La procédure de recette doit en effet être *automatique* et *reproductible*, et ne doit pas nécessiter une quelconque interprétation des résultats par l'opérateur : elle pourra alors se faire sans effort (et donc abondamment !). Cette validation sera développée *en amont* du développement du produit lui-même, et sera basée sur des jeux de test modulaires afin de permettre une validation au fil de l'eau des différents développements.

Pour vous guider dans votre appropriation de la démarche, votre mentor a déjà réalisé le travail pour cette première itération⁴ ! À cet effet, il vous fournit :

- des exemples de fiches de scénarios de validation pour la méthode `addMember()`,
- un code Java correspondant à ces scénarios (classe `addMemberTest`)⁵. Ce code s'interface avec le code d'enchaînement de tests qui vous est fourni, et qui accumule les résultats des modules de test exécutés et affiche un compte-rendu général du type : « Nombre de tests passés, nombre de tests en échec »

Vous devrez vous en inspirer lors des prochaines itérations... pour ce qui concerne votre travail du jour, vous pouvez vous contenter de comprendre les scénarios de test et de savoir que le code de test implémente ces scénarios. Si vous regardez (rapidement) le code, vous noterez combien votre mentor a été soucieux de factoriser son code de test : un code de test n'en est pas moins un code, et il convient de lui appliquer les même « bonnes pratiques » que pour tout autre code.

Phase 3 – Concevoir le logiciel attendu

Il s'agit maintenant de concevoir le logiciel qui pourra répondre aux scénarios de test prévus précédemment. Cette réflexion s'appuiera sur l'élaboration de diagrammes UML :

- Un *diagramme de classes* général (qu'on mettra à jour à chaque itération.)
- Les principaux *diagrammes de séquence détaillée* spécifiques à l'itération courante.

3 Nota : cette démarche reste valable si l'on est soi-même le client du produit qu'on développe ou s'il s'agit d'une demande interne à l'entreprise.

4 Décidément, il vous aime bien !

5 C'est principalement ce code que vous avez exécuté tout à l'heure !

L'élaboration de ces deux types de diagramme n'est pas indépendante : il y a des allers-retours fréquents entre les deux au fur et à mesure que la conception mûrit. Commencez par le diagramme de classes, avec des classes sans attribut ni méthode. Explicitez ensuite les associations entre ces classes. Vous pouvez alors ajouter des attributs (sans chercher l'exhaustivité). L'identification des méthodes de chaque classe pourra s'appuyer sur l'élaboration des diagrammes de séquences. Ce diagramme donne en effet un point de vue plus propice aux questionnements et aux choix quant aux responsabilités respectives des différentes classes, et guide ainsi la spécification des méthodes associées à ces classes. *Tout ce processus sera réitéré autant de fois que nécessaire.*

Sur le diagramme de classes, une extrémité d'association ne doit être déclarée navigable qu'à raison : toujours questionner l'intérêt de cette navigabilité par rapport au coût de gestion que cela induit (car en cas de navigabilité, il faudra gérer l'attribut correspondant). Par ailleurs, sur le diagramme de classes, l'attribut engendré par une extrémité navigable d'une association (le rôle) ne doit être affiché qu'à cet endroit et non dans la classe où il sera finalement implémenté lors du codage. Dans la même optique d'encapsulation, un attribut n'aura des accesseurs (méthodes get et set) que pour raisons dûment justifiées.

L'atteinte des objectifs de l'itération courante peut amener à remettre en cause des choix faits dans les itérations précédentes, et conduire ainsi à modifier l'existant : il faudra mettre à jour les diagrammes impactés. Mais aucune anticipation des itérations suivantes ne doit entrer en ligne de compte : *il faut rester concentré sur les exigences de l'itération courante* (tout en respectant celles des itérations passées – principe de *non régression*). Évidemment, pour cette première itération, vous n'aurez pas de problème de régression !

Vous pourrez éditer vos diagrammes UML avec votre outil de prédilection : quelques références vous sont fournies sur Moodle. Si cela a un sens pour l'outil que vous utilisez, les diagrammes que vous devez élaborer doivent faire partie du package opinion.

En plus des ressources déjà mises à disposition dans le cadre de l'UE INF111, vous pourrez également trouver des réponses à vos questions sur UML dans les références suivantes :

1. <http://uml.free.fr/index-cours.html> « Cours UML » consulté le 10 avril 2019.
2. <http://laurent-piechocki.developpez.com/uml/tutoriel/lp/cours/> « UML, le langage de modélisation objet unifié » par Laurent Piechocki. Date de publication : 22/10/07, date de mise à jour : 14/09/09, consulté le 10 avril 2019.

Phase 4 – Développer le logiciel

L'objectif à atteindre est de produire au plus simple⁶ un logiciel permettant de « passer » sans erreur les tests prévus, en s'appuyant bien évidemment sur la conception faite dans la phase précédente. On est dans une approche *minimaliste* : on ne fait que le strict nécessaire et on évite de « prévoir » de futures extensions ou les prochaines fonctionnalités. Ce qui n'empêche pas d'être exigeant sur la *qualité* du code (cf « Bonnes pratiques »). Voici notamment quelques recommandations :

- Si vous utilisez un éditeur UML qui génère du code Java, attention à ne générer que du code utile : lorsque vous créez des classes, des associations, des attributs, etc. prenez soin de choisir judicieusement les options associées de façon à éviter la production de code parasite (ne créer que les accesseurs nécessaires notamment).
- Produisez un code clair et lisible :
- Repérez et supprimez le code superflu.
- Compléter les bannières Javadoc (`/**...*/`) en utilisant les tags appropriés (cf Javadoc tag conventions <https://www.oracle.com/technetwork/articles/java/index-137868.html#tag>). La Javadoc définit l'API et permet donc d'utiliser du code existant.

- Insérer des commentaires (`/* ... */` ou `//...`) permettant aux développeurs de comprendre le code pour pouvoir le modifier.

Vu que le code de test est déjà disponible, on peut tester autant de fois qu'on le souhaite : les tests étant totalement automatisés, cela ne coûte pas grand-chose et permet de vérifier l'avancement du développement. Ne vous en privez pas !

Dès lors que plus aucune erreur n'est détectée, et dans l'hypothèse où les tests ont été bien pensés et implémentés, il est raisonnable de considérer que l'objectif de l'itération est atteint. Après un éventuel dernier petit toilettage final du code (application des « Bonnes pratiques »), vous produirez la preuve de cette victoire : la *copie d'écran du compte-rendu fourni par votre programme de test* !

Dans l'hypothèse où le temps alloué à l'itération s'achèverait avant que vous n'ayez pu atteindre ce bel objectif, le compte-rendu de test indiquera les erreurs résiduelles, qui devront de fait être traitées lors de l'itération suivante... mais attention, aux retards en cascade, le client pourrait ne pas aimer.

Phase 5 – Archiver et livrer

Voici le moment tant attendu de la production du livrable de l'itération : vous devez produire une archive (.tar.gz ou .zip) contenant la version courante de votre projet. C'est-à-dire :

- tous les fichiers de *code* (export *complet* du dossier *src* de votre projet),
- le dossier de conception (recueil des *diagrammes* UML), au format pdf
- le dossier de test (recueil des *scénarios* et *compte-rendu de test*, avec des explications pour l'éventuelle non atteinte des objectifs), au format pdf.

Vous déposerez consciencieusement ce livrable dans le dossier *livrables* de votre projet, et, le cas échéant, sur Moodle.

Phase 6 – Introspection

À chaque fin d'itération (pratique d'*amélioration continue*), vous êtes invités à discuter des problèmes rencontrés (techniques, organisationnels, humains...) par l'équipe et à prendre les décisions qui vous semblent appropriées pour vous améliorer à la prochaine itération.

Complément sur la problématique du test

À lire... et à méditer pour la suite du projet.

Une problématique centrale du test provient du choix du point de départ de l'opération : quelles sont les fonctions de l'API par lesquelles on va commencer la validation ? A priori, il convient de suspecter de dysfonctionnement l'ensemble de ces fonctions, et cette suspicion s'étend même au code de test qu'on développe ! On se retrouve donc à utiliser un outillage dont la correction est incertaine pour tester la conformité d'un produit à un cahier des charges. On mesure la difficulté de l'exercice et dès lors la nécessité d'élaborer une *stratégie* sérieuse avant de se lancer.

Quelques principes permettent de faciliter cette périlleuse mission :

1. Stratégie de test : *commencer par les fonctions les plus simples* et dont le couplage avec les autres fonctions du système est minimum. *C'est pourquoi votre mentor a choisi de démarrer par la gestion des membres, qui est complètement indépendante des autres fonctionnalités.*
2. Assurer un pas avant de passer au suivant : *voilà pourquoi votre entreprise est si friande de l'approche agile, et que votre mentor vous demande un développement itératif.*

3. S'efforcer d'examiner le plus exhaustivement possible l'espace d'états que le système peut prendre et garder à l'esprit qu'un acteur humain (notamment celui qui va utiliser le logiciel) a, *a priori*, un comportement non prédictible.
4. S'il ne faut pas oublier de tester des cas « anormaux » de fonctionnement (typiquement, les cas de levées d'exception), *il ne faut pas négliger le test des cas dits « nominaux »*. Si dans cette première itération, il y a assez peu de tests nominaux pertinents, ce ne sera pas le cas pour les itérations suivantes⁷ !

⁷ Et le point faible récurrent de vos prédécesseurs a typiquement été un manque de tests nominaux !